

Datos de imitación y modelo de atención con puntero para el WTAP

Informe: descomposición de soluciones, codificación de la entrada y de la salida (puntos 1–3)

[Nombre(s) del grupo] · Optimización combinatoria · 2026

Problema. En el Weapon-Target Assignment Problem (WTAP) hay W armas y T objetivos; el objetivo j vale V_j y el arma i lo destruye con probabilidad p_{ij} . Cada arma se asigna a lo más a un objetivo (un objetivo puede recibir varias) y se minimiza el valor esperado que sobrevive:

$$\min_x Z = \sum_{j=1}^T V_j \prod_{i=1}^W (1 - p_{ij})^{x_{ij}}, \quad q_j = \prod_i (1 - p_{ij})^{x_{ij}}$$

Llamamos q_j a la probabilidad de que el objetivo j sobreviva con las armas asignadas hasta ahora. Las instancias siguen la literatura ($V_j \sim U\{25, \dots, 100\}$, $p_{ij} \sim U(0,6; 0,9)$); se entrena con **$W = 10$ y $T = 8$** , pero el modelo acepta cualquier tamaño.

Qué cambia respecto de la Tarea 4. La MLP de la Tarea 4 asignaba las armas en orden fijo (arma 0, 1, ...) y elegía el objetivo de la arma actual. Tenía pesos distintos por posición, tamaño fijo y no podía decidir qué arma usar primero. Con un encoder de self-attention y un puntero, cada **acción posible es un par (arma $i \rightarrow$ objetivo j)** representado por una fila de la matriz de entrada, y la decisión puntual del modelo es: **dado el estado actual, ¿qué arma asigno ahora y a qué objetivo?** Es la misma pregunta que se hace MMR (Maximum Marginal Return), la heurística constructiva clásica del WTAP.

1. Descomposición de las soluciones en decisiones

Algoritmo tradicional (experto) y tipo de acciones

El experto es la **Búsqueda Local Iterada (ILS)** de la Tarea 4: parte de la solución de MMR, aplica búsqueda local best-improvement con los vecindarios reassign y swap (deltas de costo vectorizados), perturba 3 armas al azar y acepta si mejora (o con probabilidad 5 %). En instancias 10×8 alcanza el óptimo en todas las instancias verificadas con un método exacto meet-in-the-middle.

Usamos **acciones constructivas**: cada acción agrega un par (arma libre $i \rightarrow$ objetivo j) a la solución parcial. No usamos la secuencia de movimientos de búsqueda local del experto porque esa trayectoria recorre cientos de movimientos ligados a perturbaciones aleatorias y no es una política reproducible a partir del estado. Una construcción, en cambio, tiene exactamente W decisiones y cada una depende solo del estado parcial, igual que en el ejemplo del TSP. Además, representar la acción como un par (y no como "el objetivo de la arma actual") elimina la restricción de orden fijo: el modelo puede elegir qué arma usar, como MMR.

Regla de orden: de un conjunto de pares a una secuencia

La solución del experto es un **conjunto** de W pares (i, a_i^*) sin orden. Para descomponerla necesitamos decidir en qué orden se agregan los pares. Usamos el criterio de MMR: en cada paso, entre las armas aún libres, se toma el par del experto con **mayor ganancia marginal** en el estado actual:

$$i^* = \arg \max_{i \text{ libre}} V_{a_i^*} q_{a_i^*} p_{i a_i^*}, \quad \text{acción escogida} = (i^*, a_i^*)$$

La regla es determinista (una sola etiqueta por estado) y cualquier orden reconstruye exactamente la solución del experto. Como coincide con el criterio de MMR, las decisiones con grandes ganancias van primero y la política aprendida puede leerse como "MMR corregido por el experto": en la mayoría de los

pasos el par escogido es el mismo que elegiría MMR, y el modelo debe aprender a detectar cuándo no lo es.

Tuplas (estado, acciones posibles, acción escogida)

- **Estado inicial:** asignación vacía, todas las armas libres ($\text{assignment} = [-1] * W$). Es el análogo de $\text{visited}=[0]$ en el TSP.
- **Acciones posibles:** todos los pares (arma libre, objetivo), generados por `env.gen_actions(state, "constructive")`. Con k armas libres son $k \cdot T$ (80 en el primer paso, 8 en el último).
- **Acción escogida:** el par del experto que indica la regla de orden. Luego se aplica `env.state_transition(state, ("constructive", (i, j)))`, que fija $x_{ij} = 1$ y actualiza $q_j \leftarrow q_j(1 - p_{ij})$.
- **Término:** cuando las W armas están asignadas, así que hay W tuplas por instancia. A diferencia del TSP no se corta antes: incluso la última arma tiene T destinos posibles y esa decisión no es trivial.

```
def decompose(inst, expert_assignment):
    a = expert_assignment
    state = WTAP_State(inst)
    while not state.is_complete:
        free = state.free_weapons
        gains = [V[a[i]] * state.q[a[i]] * P[i, a[i]] for i in free]
        i = free[argmax(gains)]
        chosen = ("constructive", (i, a[i]))
        yield deepcopy(state), list(env.gen_actions(state, "constructive")), chosen
        env.state_transition(state, chosen)
```

Ejemplo concreto

Instancia de ejemplo (10 armas $\times$ 8 objetivos). El experto encuentra la asignación [3, 7, 2, 5, 3, 1, 6, 4, 0, 5] (arma $\rightarrow$ objetivo) con costo 52,95. Su descomposición en 10 decisiones es:

Paso	Armas libres	Acciones posibles	Acción escogida (experto)	MMR elegiría
0	10	80	arma 5 $\rightarrow$ objetivo 1	arma 1 $\rightarrow$ objetivo 1 ($\neq$)
1	9	72	arma 3 $\rightarrow$ objetivo 5	arma 6 $\rightarrow$ objetivo 3 ($\neq$)
2	8	64	arma 0 $\rightarrow$ objetivo 3	arma 6 $\rightarrow$ objetivo 3 ($\neq$)
3	7	56	arma 8 $\rightarrow$ objetivo 0	arma 8 $\rightarrow$ objetivo 0
4	6	48	arma 2 $\rightarrow$ objetivo 2	arma 2 $\rightarrow$ objetivo 2
5	5	40	arma 7 $\rightarrow$ objetivo 4	arma 7 $\rightarrow$ objetivo 4
6	4	32	arma 1 $\rightarrow$ objetivo 7	arma 1 $\rightarrow$ objetivo 7
7	3	24	arma 6 $\rightarrow$ objetivo 6	arma 6 $\rightarrow$ objetivo 6
8	2	16	arma 4 $\rightarrow$ objetivo 3	arma 9 $\rightarrow$ objetivo 3 ($\neq$)
9	1	8	arma 9 $\rightarrow$ objetivo 5	arma 9 $\rightarrow$ objetivo 5

En 4 de los 10 pasos (en amarillo) el par del experto no es el de mayor ganancia: son justamente las decisiones que MMR toma mal y que el modelo debe aprender a corregir. En todo el conjunto de validación, MMR coincide con el experto en 79,9 % de los pasos.

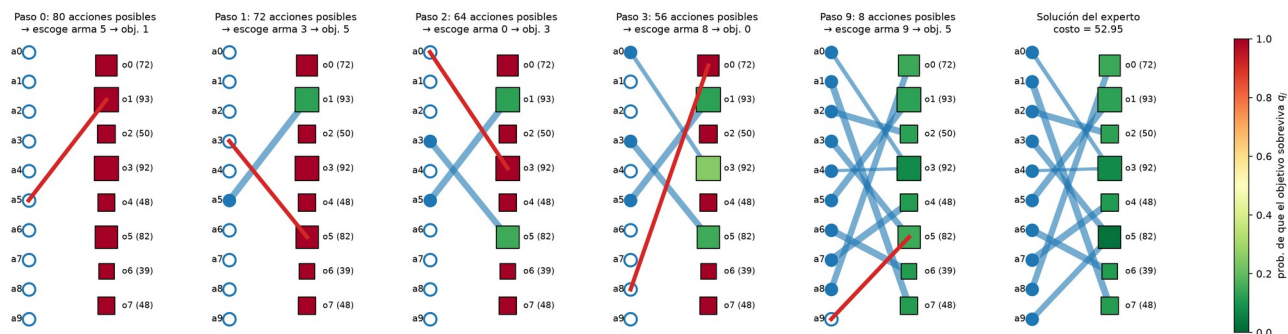


Figura 1. Descomposición de la solución del experto: en cada paso, las líneas azules son los pares ya asignados, la roja es la acción escogida y los círculos vacíos son las armas libres. Tamaño del objetivo $\propto$ valor; color = probabilidad de sobrevivir.

2. Codificación de la entrada (state2vec)

La entrada no se aplanan: es una matriz de forma $(1 + W \cdot T, 16)$ (81×16 en las instancias de entrenamiento), con dtype float32. Cada fila se procesa con los mismos pesos:

- **Fila 0: fila global** (tipo [CLS]). Resume el estado completo y forma la query de decisión. Se identifica por el flag `es_global` (columna 3) y además está siempre en la posición 0, como la ciudad inicial del TSP. Nunca es una acción válida.
- **Fila 1 + i · T + j: par (arma i → objetivo j)**. Describe a la vez el estado y la acción “asignar el arma i al objetivo j”. La correspondencia fila → par es fija durante todo el episodio.

Las columnas replican los tres roles del notebook del TSP. Las características relativas al estado se tomaron de lo que miran las heurísticas constructivas del WTAP: la **ganancia marginal** de MMR, la comparación con otras armas libres (arrepentimiento o regret) y la anticipación que se obtiene al completar la solución con MMR (que en la Tarea 4 resultó la información más valiosa). Sea F el conjunto de armas libres y $g_{ij} = V_j q_j p_{ij}$.

#	Columna	Rol	Contenido	Por qué ayuda a decidir
0	<code>p_{ij}</code>	propia	p_{ij}	efectividad del arma contra el objetivo
1	<code>valor</code>	propia	$V_j / \max V$	importancia del objetivo
2	<code>p_vs_mejor</code>	propia	$p_{ij} - \max_{i' \in F} p_{i'j} (\leq 0)$	¿es j el mejor destino posible para el arma i?
3	<code>es_global</code>	flag	1 solo en la fila 0	marca la fila que construye la query de decisión
4	<code>no_disponible</code>	flag	1 si el arma i ya fue asignada, o si es la fila global	MÁSCARA: logit -10^9
5	<code>asignado</code>	flag	1 si $x_{ij} = 1$ en la solución parcial	qué pares ya se eligieron (contexto)
6	<code>q_sobrevive</code>	relativa	q_j	cuánto queda por destruir en j
7	<code>valor_en_juego</code>	relativa	$V_j q_j / \max V$ (fila global: $\sum V q / \sum V$)	valor que aún se puede ganar en j
8	<code>ganancia *</code>	relativa	$g_{ij} / \max g$ sobre pares disponibles	criterio de MMR (análogo a dist_a actual)
9	<code>es_mmr</code>	relativa	1 si (i, j) es el par que elegiría MMR	elección de la heurística clásica
10	<code>armas_en_obj</code>	relativa	n° de armas ya asignadas a j / W	rendimientos decrecientes
11	<code>ventaja</code>	relativa	$p_{ij} - \text{media}_{\{i' \in F\}} p_{i'j}$	¿es esta arma comparativamente buena para j?
12	<code>arrepentimiento</code>	relativa	$(g_{ij} - \max_{\{i' \in F, i' \neq i\}} g_{i'j}) / \max g$	¿otra arma libre aprovecharía mejor j?
13	<code>mmr_completa *</code>	relativa	1 si al completar con MMR el arma i va a j	anticipación de decisiones futuras
14	<code>q_final_mmr</code>	relativa	q_j al completar con MMR	¿quedará j bien cubierto al final?
15	<code>frac_restantes</code>	global	$ F / W$ (repetida en todas las filas)	etapa de la construcción

- **Query de decisión:** la fila marcada con `es_global` (columna 3). En el WTAP no hay una “ciudad actual”: el estado es la solución parcial completa. Tras la self-attention, la fila global concentra

información de todos los pares. La query se completa con el promedio de los embeddings de los pares disponibles (sección 3).

- **Máscara:** no_disponible (columna 4). Vale 1 en la fila global y en los T pares de cada arma ya asignada. Así, con k armas libres quedan exactamente k·T filas válidas, las mismas acciones que genera el entorno.
- **Información global:** se repite en todas las filas (frac_restantes) y también se guarda en la fila especial (valor_en_juego total). Las características relativas solo se calculan para pares disponibles; en las filas enmascaradas valen 0.
- **Independencia del índice y del tamaño:** ninguna columna contiene el número de arma ni de objetivo, y todas están normalizadas por instancia (divididas por el máximo o expresadas como fracciones). Por eso el mismo modelo procesa 81, 151 o 301 filas. La relación entre pares (por ejemplo, otra arma mejor para el mismo objetivo) puede además aprenderse con la self-attention, porque el par (i, j) puede atender a los pares (i', j).

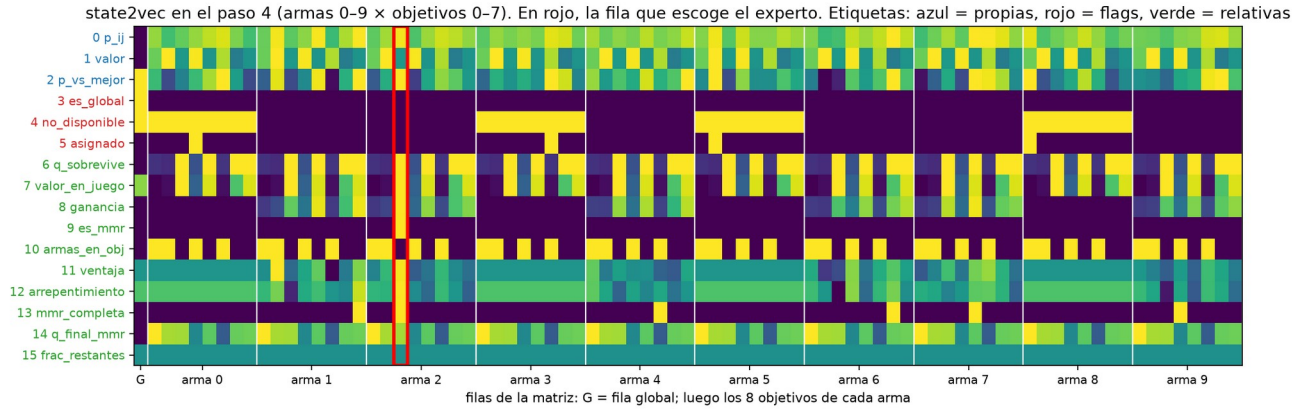


Figura 2. state2vec del paso 4 del ejemplo (matriz transpuesta: cada columna del gráfico es una fila de la matriz; G = fila global; luego los 8 objetivos de cada arma). En rojo, la fila que escoge el experto. Cada característica se reescaló a [0, 1] solo para visualizarla.

3. Codificación de la salida

El modelo entrega un **vector de logits de largo $n = 1 + W \cdot T$** (uno por fila) y un softmax lo convierte en la probabilidad de que cada fila sea la próxima acción:

$$u_r = \frac{(W_Q^{ptr} q) \cdot (W_K^{ptr} h_r)}{\sqrt{d}}, \quad u_r = -10^9 \text{ si } x_{r,4} = 1, \quad P(r) = \text{softmax}(u)_r$$

- **Acción ↔ fila:** la acción ("constructive", (i, j)) es "elegir la fila $r = 1 + i \cdot T + j$ ", y a la inversa $i = (r - 1) // T$, $j = (r - 1) \% T$. Aunque la acción involucra dos elementos (un arma y un objetivo), cada par tiene su propia fila, así que el puntero lo selecciona directamente sin combinar filas, como sí haría falta en un movimiento 2-opt. El agente consulta $\text{probs}[1 + i \cdot T + j]$ para cada acción legal.
- **Filas enmascaradas:** las que tienen no_disponible = 1 (columna 4), es decir, la fila global y todos los pares de armas ya asignadas. Reciben logit -10^9 antes del softmax, por lo que su probabilidad es exactamente 0 y no compiten con las acciones válidas.
- **Etiqueta Y:** el índice entero de la fila escogida por el experto, $Y = 1 + i \cdot T + a_i^*$. Equivale a un vector one-hot de largo n (1 en la acción escogida y 0 en el resto), pero se guarda como entero (tensor long) para usar CrossEntropyLoss:

$$\mathcal{L} = -\frac{1}{N} \sum_{s=1}^N \log P_s(Y_s), \quad Y_s = 1 + i \cdot T + a_i^*.$$

Por la máscara, el softmax se reparte solo entre las k·T acciones válidas; un modelo sin entrenar parte con una pérdida cercana a log(k·T) en promedio. Como la etiqueta nunca está enmascarada (se verifica con un assert sobre todas las muestras), la pérdida siempre es finita.

Cómo se construye q: adaptación de TSP_Attention

La estructura del notebook se mantiene (embedding compartido → self-attention con residual → feed-forward → query → puntero). Cambian la query y la máscara:

q = W_c [h_global ; h_disp], h_disp = (sum_r (1 - x_{r,4}) h_r) / (sum_r (1 - x_{r,4}))

	TSP_Attention (notebook)	WTAP_Attention (esta tarea)
Filas	ciudades	fila global + pares (arma, objetivo)
VEC_LEN	6	16
Encoder	1 bloque	2 bloques con LayerNorm y Dropout, d = 64
Query (paso 3)	Wc[h_actual ; h_inicial] (col. 2 y fila 0)	Wc[h_global ; promedio de disponibles] (col. 3)
Máscara (paso 4)	visitada (col. 3)	no_disponible (col. 4)
Puntero	(W_Q q)·(W_K h_i)/√d	igual

Resultado (puntos opcionales 7–8, detallados en el notebook). Entrenado con 60.000 pares (estado, fila) de 6.000 instancias, el modelo coincide con el experto en 87,3 % de las decisiones de validación. Usado como política de un GreedyAgent, logra un gap de 2,97 % respecto del experto en las mismas 50 instancias de prueba de la Tarea 4 (MMR: 7,68 %; MLP de la Tarea 4: 4,42 %), y funciona sin cambios en instancias más grandes (10×8: 3,0 %, 15×10: 4,9 %, 20×15: 3,9 %).

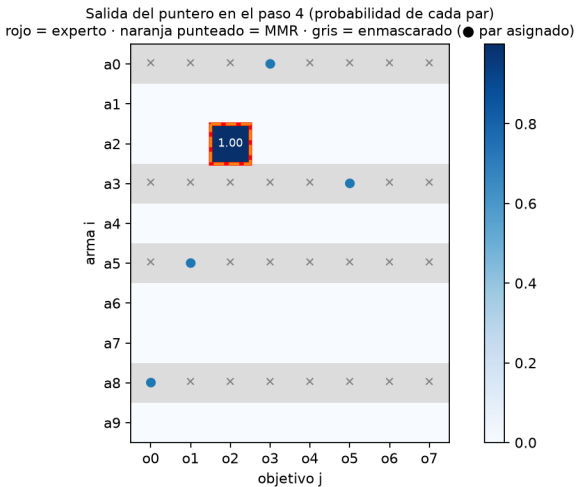


Figura 3. Salida del puntero entrenado en el estado de la Figura 2, reordenada como armas × objetivos. Gris = filas enmascaradas; rojo = acción del experto; naranja = MMR. El modelo pone probabilidad 1,00 en el par del experto, que aquí coincide con MMR.

Referencias

den Broeder, G. G., Ellison, R. E., & Emerling, L. (1959). On optimum target assignments. Operations Research, 7(3).
Lloyd, S. P., & Witsenhausen, H. S. (1986). Weapons allocation is NP-complete. Summer Computer Simulation Conference.
Ahuja, R. K., Kumar, A., Jha, K. C., & Orlin, J. B. (2007). Exact and heuristic algorithms for the weapon-target assignment problem. Operations Research, 55(6).
Vinyals, O., Fortunato, M., & Jaitly, N. (2015). Pointer Networks. NeurIPS.
Kool, W., van Hoof, H., & Welling, M. (2019). Attention, Learn to Solve Routing Problems! ICLR.
Notebook del curso: “Atención para el TSP: de la MLP al puntero”.